

ROBOTICS INTEGRATION GROUP PROJECT I

Experiment 3

Desktop Object Automatic Classification and Sorting

ROS 2, Gazebo and MoveIt simulation

Student	Zihan Wang
Student ID	24020036065
Personal role	Simulation integration and motion-flow collaboration
Task classes	0: green original; 1: yellow pineapple
Runtime	ROS 2 Humble, Gazebo, MoveIt 2
Report date	27 September 2026

This individual report is based on the E3 simulation source code, configuration, launch files, workflow record, screenshots and demonstration material supplied with the project.

Contents

1	Basic Requirements of the Experiment	2
2	Personal Division of Work	2
3	Implementation Principles	3
3.1	System architecture and one-command startup	3
3.2	Perception, filtering and grid assignment	4
3.3	Queue scheduling and ROS 2 Action	4
3.4	Motion sequence and bin allocation	5
4	Problems Encountered and Solutions	5
5	Verification Evidence and Result	5
6	Personal Repository and Commit Traceability	6
7	Conclusion	7

1 Basic Requirements of the Experiment

The objective of Experiment 3 is to build an autonomous desktop-object sorting system. The system must acquire an overhead RGB image, detect objects, map each detection to a fixed desktop grid, command a robot to pick the object, and place it in the correct classification area. For this task, the two semantic classes are **original-flavor green milk cartons (class 0)** and **pineapple-flavor yellow milk cartons (class 1)**.

The simulation design addresses the following requirements:

- at least two object classes, six fixed pick grids, and two placement bins;
- image input through ROS 2 and structured detection messages containing class, bounding box and confidence;
- automatic selection of the nearest calibrated grid rather than manual selection of a target;
- a feedback-capable ROS 2 Action for serial pick-and-sort execution;
- no manual class, grid or placement decision after the task begins;
- collision-aware motion planning, a safe clearance pose, and controlled gripper attachment/detachment;
- handling of missing or unstable detections, motion-planning failures, attachment timeouts and uncertain object state;
- one launch command that starts the simulator, planning stack, detector, action server and task manager.

The supplied project implements a fixed-workspace solution. It uses calibrated grid centers and pre-validated joint configurations, which is appropriate for repeatable desktop sorting and avoids requiring arbitrary pose grasping.

2 Personal Division of Work

I, **Zihan Wang**, was responsible for **simulation integration and motion-flow collaboration**. My work focused on connecting the perception, grid-assignment, task-queue and robot-execution modules into one serial process. The key contributions were:

- checking the interface from visual detections to the task manager, including confidence filtering, class validation and nearest-grid assignment;
- participating in the design of the stable observation stage, where a complete six-grid result must remain consistent over consecutive frames before a queue is frozen;
- coordinating the action-level execution sequence: approach, local centering, gripper close, attachment confirmation, fixed lift, common clearance, bin placement, release and safe retreat;
- checking the coordination between MoveIt planning, trajectory controllers, grid-specific joint poses and the left/center/right bin slots;
- enforcing serial execution: only one object may be handled at a time, and a failed action halts the round instead of advancing the queue;
- documenting simulation issues and the corresponding safety-oriented solutions for subsequent RoboMaster EP deployment reference.

The detection-model training, real-robot data collection, calibration and deployment were completed by other group members. This report therefore concentrates on the simulation integration work and does not claim individual ownership of those parts.

3 Implementation Principles

3.1 System architecture and one-command startup

The complete simulation stack is started from the ROS 2 workspace after sourcing the Humble installation and the local overlay. The command used for the supplied project is shown in Figure 1.

```
<> Bash

cd ~/robot-integration-project-1/E3_desktop_sorting/ros2_ws

source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch e3_sorting_system e3_sorting.launch.py
```

Figure 1: One-command startup of the E3 desktop-sorting system.

The top-level launch file includes the Gazebo simulation and the MoveIt `move_group` launch, then creates the detector, pick Action server and task-manager nodes. The Action server is configured for simulation time and enables the Gazebo attachment plugin. This organization avoids a fragile multi-terminal workflow and makes the system boundary explicit (Figure 2).

```
e3_sorting.launch.py x
1 | """Formal E3 system: include the simulator once and gate observation."""
2
3 import os
4
5 from ament_index_python.packages import get_package_share_directory
6 from launch import LaunchDescription
7 from launch.actions import IncludeLaunchDescription
8 from launch.launch_description_sources import PythonLaunchDescriptionSource
9 from launch_ros.actions import Node
10
11
12 def generate_launch_description():
13     share = get_package_share_directory("e3_sorting_system")
14     moveit_share = get_package_share_directory("mecharm_270_moveit_config")
15     config = os.path.join(share, "config", "sorting_config.yaml")
16
17     def include(directory, filename):
18         return IncludeLaunchDescription(PythonLaunchDescriptionSource(
19             os.path.join(directory, "launch", filename)))
20
21     return LaunchDescription([
22         include(share, "e3_sim.launch.py"),
23         include(moveit_share, "move_group.launch.py"),
24         Node(package="e3_sorting_system", executable="object_detector",
25             name="e3_object_detector", output="screen",
26             parameters=[config, {"use_sim_time": True}]),
27         Node(package="e3_sorting_system", executable="pick_action_server",
28             name="e3_pick_action_server", output="screen",
29             parameters=[config, {
30                 "use_sim_time": True, "dry_run": False,
31                 "classification_transport_enabled": True,
32                 "attach_enabled": True, "lift_calibration": False,
33                 "direct_pick_calibration": False,
34                 "local_center_calibration": False,
35                 "bin_slot_calibration": False,
36                 "grid_pick_calibration": False,
37                 "calibration_execute": False}],),
38         Node(package="e3_sorting_system", executable="task_manager",
39             name="task_manager", output="screen",
40             parameters=[config, {"use_sim_time": True}]),
41     ])
```

Figure 2: Key part of the unified ROS 2 launch file.

The operational data flow is:

```
Gazebo overhead camera → object_detector → /e3/detections → task_manager →
/e3/pick_and_sort Action → pick_action_server → MoveIt and joint-trajectory
controllers
```

3.2 Perception, filtering and grid assignment

Each detected box contains a category hypothesis, confidence and center position. The task manager first selects the highest-scoring hypothesis, then rejects invalid numerical values, unsupported categories and scores below the configured confidence threshold. A detection is assigned to the calibrated grid whose pixel center has the smallest Euclidean distance:

$$d_i = \sqrt{(u - u_i)^2 + (v - v_i)^2}. \quad (1)$$

The detection is accepted only when the closest distance is within the assignment limit, and when its category is consistent with the expected class for that grid. The code excerpt in Figure 3 records the three important decisions: filtering, nearest-grid matching and queue freezing after stable observations.

```

37
38 def observe(self, detections, now):
39     if self.state != "observing":
40         return
41     if now >= self.deadline:
42         self.tick(now)
43         return
44     assigned = {}
45     duplicates = set()
46     for detection in detections:
47         if not detection.results:
48             continue
49         hypothesis = max(
50             (r.hypothesis for r in detection.results),
51             key=lambda h: h.score)
52         label, score = hypothesis.class_id, hypothesis.score
53         point = detection.bbox.center.position
54         if (label not in ("comb", "mouse")
55             or not all(math.isfinite(v)
56                       for v in (point.x, point.y, score))
57             or score < self.confidence):
58             continue
59         grid = min(self.centers, key=lambda g: math.dist(
60             self.centers[g], (point.x, point.y)))
61         distance = math.dist(self.centers[grid], (point.x, point.y))
62         if distance > self.distance:
63             continue
64         if label != ("comb" if grid % 2 else "mouse"):
65             continue
66         if grid in assigned:
67             duplicates.add(grid)
68         assigned[grid] = label
69     for grid in duplicates:
70         assigned.pop(grid, None)
71     complete = set(assigned) == set(self.centers)
72     same = complete and assigned == self.last
73     self.count = self.count + 1 if same else (
74         1 if complete else 0)
75     self.last = assigned
76     if self.count >= self.frames:
77         self.queue = [(g, assigned[g]) for g in sorted(self.centers)]
78         self.state = "ready"
79         self.message = "All grids stable; queue frozen for this round"
80

```

Figure 3: Task-manager logic for detection filtering, nearest-grid matching and stable-queue freezing. The legacy internal label strings are implementation details; this report uses the milk-carton class definitions stated on the title page.

The configuration requires a minimum confidence of 0.70, a maximum assignment distance of 45 pixels and three consecutive stable frames. These checks prevent one weak or shifted detection from triggering a physical motion.

3.3 Queue scheduling and ROS 2 Action

After all six calibrated grids are present and stable, the task manager freezes a queue ordered by grid identifier. It sends one `PickAndSort` goal at a time, containing the grid identifier, object class and simulation model identifier. The next goal is sent only after the current Action returns success. If a goal is rejected, cancelled or fails, the remaining queue is not executed.

This policy is intentionally conservative. It protects against a common failure mode in automation: treating a failed grasp as if the object had been moved and then continuing with an incorrect scene model.

3.4 Motion sequence and bin allocation

The six grids have separate pick and lift joint configurations. After a successful grasp, the robot follows a grid-specific lift configuration, a common high-clearance configuration, an approach configuration for the destination bin, and a left/center/right release configuration. Green original-flavor cartons are routed to the green-class bin, while yellow pineapple-flavor cartons are routed to the yellow-class bin. The column position determines the release slot.

For each object, the sequence is:

Open gripper → fixed pick pose → local centering correction → close gripper → attach/check → fixed lift → common clearance → bin approach → slot release → detach/open gripper → safe retreat

Gazebo attachment services are used because gripper friction alone does not reliably hold the simulated cartons. Attachment is accepted only after the gripper has closed and the object is sufficiently close to the grasp link. The process is therefore coupled with follow checks before transporting an object.

4 Problems Encountered and Solutions

Problem	Analysis and individual reasoning	Solution adopted
Repeated inverse-kinematics failure during segmented lifting	Small Cartesian lift increments required fresh IK solutions. Some positions were close to singular or unreachable configurations, so failure probability accumulated across segments.	Replaced long online segmented lifting with calibrated, grid-specific <code>lift_joints</code> . MoveIt still validates the goal and plans the trajectory; local IK is retained only for millimetre-scale centering correction.
Camera occlusion by the arm	In the default pose, the arm blocked the middle grids and only four targets could be observed. Starting the queue from incomplete evidence would produce a false scene model.	Added a controlled observation pose before detection. The manager waits for six stable grid assignments before freezing the queue.
Gazebo attachment timeout	Attachment plug-in or simulation-clock stalls can leave the object state uncertain. Blind retrying could produce duplicate constraints or move a detached object.	The Action records the failed stage, stops the round and enters a recovery-required state. It does not advance to the next grid under uncertainty.
Objects disturbed during transport	A low route after grasping could pass close to unprocessed objects.	Raised the common clearance configuration and used a fixed safe retreat after release.
Queue progression after a failed grasp	Continuing after a failure would make later placement decisions unreliable and hide the failed grid.	Enforced serial execution and stop-on-failure semantics; only a confirmed successful Action allows the next goal.

Table 1: Simulation issues and the safety-oriented solutions.

The recorded test history includes an attachment timeout at grid 6. The key result is not that a timeout disappeared, but that the system did not continue with an uncertain object state. The earlier grids had completed serial handling before the protected stop. This behavior is aligned with the experiment requirement for exception handling and safe termination.

5 Verification Evidence and Result

The supplied project includes the Gazebo world, six grid definitions, two bins, an overhead camera, ROS 2 interfaces, the pick-and-sort Action, configuration files, unit tests and a formal launch file. The detector screenshot in Figure 4 is retained as visual evidence of the image-recognition stage.

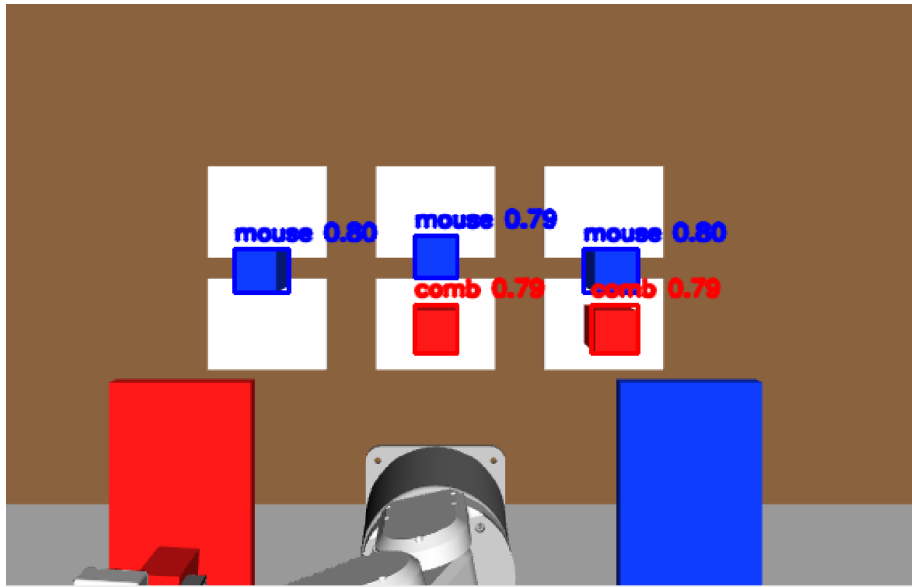


Figure 4: Saved detector-stage evidence from the E3 simulation package.

The implementation meets the intended design of two-class fixed-grid sorting, automatic queue construction, serial Action execution and safe failure handling. The available evidence should be described precisely: a complete six-object uninterrupted recording must be verified again before making a stronger claim than the recorded five completed grids followed by a safe stop at the sixth attachment timeout.

6 Personal Repository and Commit Traceability

The complete code, report source, screenshots and relevant exercises will be uploaded to my personal repository:

<https://github.com/HesterWang0419/E3-desktop-sorting>

The repository is organized as meaningful development stages rather than one bulk upload:

1. add the ROS 2 interfaces, simulation world and initial system structure;
2. add perception-to-grid scheduling and stable queue management;
3. add fixed-lift motion, bin-slot routes and failure safeguards;
4. add launch instructions, tests and workflow documentation;
5. add this English individual report and the commit-history evidence.

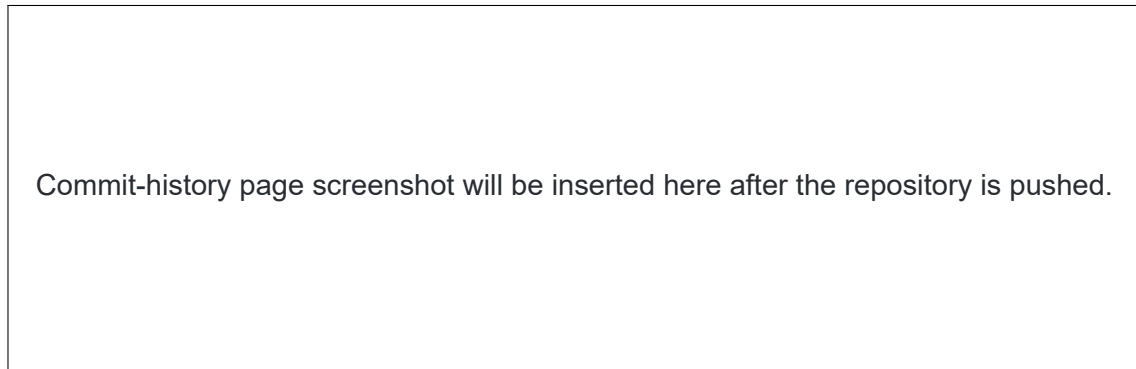


Figure 5: Reserved location for the required GitHub/Gitee commit-history screenshot.

7 Conclusion

This project applies ROS 2, Gazebo, MoveIt and a camera-based detector to a fixed-grid desktop sorting task. My simulation contribution was to integrate the perception, assignment, queue and robot-motion stages into a conservative serial workflow. The most important design choice was replacing unstable repeated IK lifting with verified fixed lift configurations, while preserving local correction and MoveIt state validation. Together with stable-frame confirmation, attachment checks, clearance routes and stop-on-failure behavior, the system provides a reproducible simulation basis for the subsequent RoboMaster milk-carton task.